
mkpkg Manual

Release 8.1.2

Gene C

Oct 09, 2026

MKPKG MANUAL:

1	mkpkg	1
1.1	Synopsis	1
1.2	Overview	1
1.3	Triggers Introduction	2
1.4	Background Motivation	3
1.5	Getting Started	3
1.6	Options	4
2	Triggers	7
2.1	_mkpkg_depends	7
2.2	_mkpkg_depends_files	8
2.3	Variable _dep_vers_prog	9
3	Appendix	11
3.1	Metadata Files	11
3.2	How mkpkg works	11
3.3	Installation	12
3.4	Dependencies	12
3.5	Some history	12

1.1 Synopsis

Tool to rebuild Arch packages triggered by changes in specified dependencies.

1.2 Overview

By default Arch packages are rebuilt when the package version changes. This happens when the tool has an update or by incrementing the package “release” number which forces a rebuild.

It would be helpful to rebuild when something the package depends on has changed in a way that requires a rebuild. This might be a new shared library or a major version bump in something that is used by the package. It could be a systemd service file provided by the packager.

This is where *mkpkg* can be useful. It helps automate rebuilds when something the package depends on has changed in a way that requires a fresh build, even when the package itself has not changed.

mkpkg uses a list of triggers that it uses to decide when a rebuild is required. Some triggers are automatic like shared library soname versions while others are provided by the packager.

The list of provided triggers is read from the variable *_mkpkg_depends* in the PKGUILD. This should contain a list of files or packages. When *mkpkg* detects a change in one or more of these it bumps the package release number and rebuilds the package.

Of course if there is a change in the actual package itself, then the package is rebuilt anyway for the new version, independent of any changes in dependencies.

If you have ever needed to rebuild a package by manually bumping the release version, then something is less than ideal. If something requires a rebuild, other than the package itself having an update, it should happen automatically.

A rebuild is triggered when a trigger file changes or if a package changes. package changes can be limited by its version. For example the trigger:

```
'python>major'
```

causes a rebuild when Python’s major version changes. Similarly:

```
'python>minor'
```

triggers a rebuild when the minor version changes.

Once run is complete mkpkg outputs:

```
mkp-status: <status> <package-version>
```

Which is designed to be easily parsed programmatically. Useful when tools are used to run all the builds. Status is one of current, success, or error. Current means package is up to date while success means package was built successfully.

1.2.1 Documentation

The manual, in both HTML and PDF formats, is installed under `/usr/share/mkpkg/docs`. and also available at: [readthedocs](#).

1.2.2 Signed Source

All git tags are signed with arch@sapience.com key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature using `git tag -v <tag-name>`

1.3 Triggers Introduction

Triggers are discussed in detail in *Triggers*.

mkpkg allows you to define a set of event rules that trigger a rebuild. A rule might be a new version of a package or a change in a file.

The packager is responsible for providing the list of triggers. Other than changes in *sonames*, which are handled automatically.

Triggers are given by a PKGBUILD array variable.

This example leads to a rebuild if either systemd or openssl is updated:

```
_mkpkg_depends=(openssl systemd)
```

While this one only rebuild when python package has a minor (or major) version change:

```
_mkpkg_depends=('python > minor')
```

In this example changes to the service file trigger a rebuild:

```
_mkpkg_depends_file=('xxx.service')
```

Thus ensuring that the new service file is packaged.

An additional little benefit, if packages are up to date then running mkpkg is significantly faster than makepkg; it can be something like 10x faster or even more.

1.4 Background Motivation

mkpkg has one run-time dependency, python.

It uses makepkg to perform the actual package builds in the usual way. That said, makepkg is a part of pacman which is always installed and thus not a *dependency* as far as PKGBUILD is concerned.

When a tool chain used to build a package is updated, it's good practice, IMHO, to rebuild packages which use that tool chain. For example, when gcc, cargo, binutils et al are updated packages using those tools should also be updated. It is standard practice to rebuild and test kernel packages when the build toolchain is updated. The same is true for many other packages.

This ensures that things compile and work properly with the new build tools and may be critical in reducing an attack surface. A little example, as of time of writing, not to pick on cargo, is [CVE-2022-36113](#)

Of course this would require a case where cargo is actually downloading something which should never be permitted; still, it's a conceivable danger.

While static linked libraries surely don't demand a rebuild to function, obviously, because the older library is part of the binary itself, it's still a good idea to rebuild it. This will pick up bug fixes, including security related ones, as well as improvements. Of course, it's always sensible to confirm that an application properly builds and works with the newer tool or library as well.

It is also helpful to know as early as possible that a package continues to build and pass all it's tests when things it depends on change.

Obviously it makes more sense to set up the rules that should lead to rebuilding a package rather than relying on human judgement. Since the triggers are known in advance it makes perfect sense to automate this.

A small comment on shared libraries. While these are generally not a problem, there is an assumption that the library itself still functions the same for whatever part of it the tool is using.

The majority of providers are careful with *sonames* as well, so most of the time that's likely true, however, the cautious among us may want to run regression tests even in this case.

Certainly for mission critical tools. Bugs happen, and it's good to learn about any issues as soon as possible.

But there are indeed some shared library packages, some with dynamically loaded libraries (plugins) that may also be trigger packages. One symptom of that need are those packages that are manually rebuilt by forcing a release version bump typically with a comment such as *rebuilt with latest ...* - we see plenty of that happening.

1.5 Getting Started

Edit the PKGBUILD and add a `_mkpkg_depends` variable with a list of triggers that should cause a rebuild when the condition is met. Triggers are discussed in detail ([Triggers](#)), but a simple example is:

```
_mkpkg_depends=('python>major', 'python-foo')
```

This would trigger a package rebuild if a version of *python-foo* is installed more recently than the last package build or if *python* has a major version which is larger than that used when package was last

built.

With the trigger conditions in the PKGBUILD, then simply call `mkpkg` instead of `makepkg`. Couldn't be simpler. Options for `mkpkg` are those before any double dash `--`. Any options following `--` are passed through to `makepkg`.

1.6 Options

The options currently supported by `mkpkg` are:

```
positional arguments:
  makepkg              All args after -- passed to makepkg.

options:
  -h, --help          show this help message and exit
  -f, --force          Bump package release and rebuild
  -r, --refresh        Update saved metadata files.
  -so-comp, --soname-comp COMP soname rebuilds never, newer, keep, major/minor/last (keep)
  -v, --verb          More verbose output - shows output of makepkg.
```

1.6.1 Config file

Configs are looked for in first in `/etc/mkpkg/config` and then in `~/.config/mkpkg/config`. Config files are in TOML format. Config options are the same as command line options with `-` replaces by `_`.

e.g. to change the default soname rebuild compare option to newer:

```
soname_comp = "newer"
```

1.6.2 Note on refresh

Attempts to update saved metadata files. Faster, if imperfect, alternative to rebuild. If there is no saved metadata, and build is up to date, will try refresh the build info. See the [Metadata Files](#) section in the Appendix.

Note that *sonames* are found by examining any executables in the *pkg* directory. If the *pkg* directory is empty, the refresh will not find any sonames.

1.6.3 Note on -soname-comp

How to handle automatic soname changes. Default value is *keep* - only rebuilds if soname is no longer available.

- *newer*
if soname is newer then rebuild (time based)
- *keep*
if soname library is still available, then dont rebuild even if newer version(s) are available
- *vcomp*
rebuild if soname version is greater than the *vcomp* version. *vcomp* is one of *major*, *minor*, *patch*, *extra* or *last* - same as for regular dependencies.
- *neverever*
Developer option - will not rebuild even if the soname library is no longer available.

1.6.4 Note on passing options to makepkg

All options following a double dash – are passed through to makepkg

TRIGGERS

There are 2 kinds of triggers. A trigger based on package and a trigger based on a file. Each of these uses a PKGBUILD variable with an array of triggers.

2.1 `_mkpkg_depends`

This is used to configure packages. Changes in one of the packages that meets the condition of the trigger, will cause a rebuild.

The variable in PKGBUILD is: `_mkpkg_depends`

This holds an array of package based triggers. Each trigger in the list can take one of 2 forms:

- **package_name**
This item is the name of a package. Rebuild is triggered if the install time of this package is newer than the the last build time.
- **package_name compare-op vers_trigger**
This is a semantic version trigger. Versions are of the form 'major.minor.patch' or more generally 'elem1.elem2.elem3....' White space around the comparison operator is optional. Rebuild is triggered when the version of *package_name* satisfies the rule.
e.g. 'python > minor' causes a rebuild when the minor version is larger than the minor version used in previous build.

Where:

- *compare-op*
is one of : >, >= or <
- *vers_trigger*
Based on comparing the first [N] elems of the version or the entire version.
 - First_[N] : rebuild if first [N] elems of package version greater than when last built
 - major : alias for First_1 (rebuild if major > last_build)
 - minor : alias for First_2 (rebuild if major.minor > last_build)
 - patch : alias for First_3 (if major.minor.patch > last_build). Also called *micro*
 - extra : alias for First_4 (major.minor.patch.extra) Also called *releaselevel*
 - serial : alias for First_5 (major.minor.patch.extra.serial)
 - last : rebuild if package version > last_build version.

last is very similar to a time based trigger but based on version instead of time.

For example, the expression

```
'pkg_name>First_2'
```

or equivalently:

```
'pkg_name>minor'
```

Say the current package version is 1.2.3, and the version when last built was 1.2.0 then the versions being compared would be

```
'1.2' > '1.2' which is false.
```

Whereas if the expression was:

```
'pkg_name>First_3'
```

then the comparison would be

```
'1.2.3' > '1.2.0'
```

which is true

N.B. The package must be built at least once using mkpkg so it can save the various package versions used. So if a version trigger is added, then this triggers a rebuild as it treats this as if the dependent package version is greater than last used (which is not known at this point). On subsequent builds the last built version of each dependent package is then known.

Unlike the standard *makedepends* variable, this allows one to not include things that are required to build the package but don't have any affect on the tool function. For example 'git' - which while required to build will not generally change the tool.

Another example, if python was version 3.13 when the package was last built and we have::

```
_mkpkg_depends=('python>minor' 'python-dnspython')
```

Then a rebuild will be done if python is greater than or equal to 3.13.x or if python-dnspython was installed more recently than the last build. This will not trigger a rebuild if python is updated from 3.13.7 to 3.13.8, since this is a patch update not a minor or major update.

Why support '<' you may ask. The only sensible use for less than operator would be to provide a mechanism to trigger a rebuild when a package gets downgraded. This would be accomplished using

```
pkg_name < last
```

2.2 **_mkpkg_depends_files**

The variable **_mkpkg_depends_files** is used to track changes in files. Filenames are relative to the directory containing PKGBUILD.

This might be useful, for example, if the source for some daemon doesn't provide a systemd service file, and the packager adds one. If that file is in `_mkpkg_depends_files`, then changes to the file trigger rebuilds.

These variables offer considerable control over what can be used to trigger rebuilds.

For example:

```
_mkpkg_depends_file=('xxx.service')
```

This directs a rebuild when the file `xxx.service` changes.

2.3 Variable `_dep_vers_prog`

In cases where a dependent package being used to build against is not yet installed, then it's version cannot be extracted by calling `pacman -Qi`. Instead, a script can be provided in `PKGBUILD` which returns the version of the package name provided as an argument.

For example if we have the following dependencies where `foo` and `goo` are used but not yet installed on the build machine (at least not the version being built against):

```
_mkpkg_depends=('python>minor', 'openssl>3.0', 'foo', 'goo')

declare -A _dep_vers_prog
_dep_vers_prog['foo']='./dep-vers'
_dep_vers_prog['goo']='./dep-vers'
```

where the script `./dep-vers` returns the version of it's one argument - where arguments can be `foo` or `goo`.

One example is `pigeonhole` and `dovecot`. This are in separate git repos and built as separate packages. `pigeonhole` is built against the just compiled version of `dovecot`. In this case a script which extracts `pkgver` from the `.PKGINFO` file either in the `dovecot pkg` directory, or by extracting it from the just built package does the trick.

3.1 Metadata Files

mkpkg keeps metadata information in files. These files are in the same directory as the PKGBUILD file:

- **.build-info**: last version built
- **.build_time**: time and status of last build
- **.mkpkg_dep_soname**: list of all shared libraries and sonames
- **.mkpkg_dep_vers**: list of trigger packages and versions last built against

3.2 How mkpkg works

Outline of what it does

- If PKGBUILD has a `pkgver()` function, check if the `pkgver` variable matches its output
- If the 2 `pkgver` match or if there is no `pkgver()` function then check if a matching package exists
- If package not up to date, then run `makepkg build`.
- If package seems otherwise up to date, then check if any of the conditions given by *mkpkg_depends* or *mkpkg_depends_files* triggers a build. If a build is called for, then bump the `pkgrel` and rebuild.
- If the package is out of date, as there is newer version then reset `pkgrel` back to “1” and build.

So, if a package builds and gets larger package release number, it was because of some trigger package dependency; absent manual modification. If package release is “1” - then you know its a fresh package version.

I use separate tool to run all my package builds so I prefer the output to be easily parseable and provide simple and clear information to feed the builder too.

mkpkg thus prints a line of the form:

```
*mkp-status: <status> <package-version>*
```

Where status is one of :

- **current** -> package is up to date
- **success** -> package was built successfully
- **error** -> problem occurred.

Obviously, package-version is what is sounds like.

It is possible for mkpkg itself to fail for some reason, in which case the *mkp-status:* line could be absent. This is also simple to detect programatically.

3.3 Installation

Available on

- [Github-mkpkg](#)
- [Archlinux AUR](#)

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. All git tags are signed with arch@sapience.com key which is available via WKD or download from <https://www.sapience.com/tech>. Add the key to your package builder gpg keyring. In PKGBUILD use source= line with *?signed* at the end. You can also manually verify the signature

To build manually, clone the repo and:

```
./scripts/do-build  
./scripts/do-install <destination-directory>
```

3.4 Dependencies

- Run Time: - python (3.14 or later) - pyalpm - python-tomli-w - python-pyelftools - python-pyconcurrent
- Building Package :
 - git
 - meson
 - meson-python
 - rsync
 - bash

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2022-present Gene C <arch@sapience.com>

3.5 Some history

3.5.1 Version 6.0.0

- soname rewrite
New argument for how soname changes are treated : *-so-comp*, *-soname-comp*.
Can be *<compare>*, *newer*, *never* or key how to compare the soname versions. The comparison types are the same as for package dependencies described above. Default is *last* which means the

entire soname version will be compared to what's available and rebuild will be triggered if a later version is now available.

<compare> e.g. >major or >minor' or last etc. If the last built soname was 5.1, and now available is 5.2 then *minor* and *last* will trigger rebuild while *major* would not. *newer* triggers if the last modify time of the library is newer.

Previous version used sonames produced by makepkg - however this only generates sonames if they are listed as dependencies. We want to get every soname - so we started over from scratch. By using our own soname generator we catch every soname and its absolute path - this enables us to correctly treat soname changes. This approach will also correctly deal with any *rpath* loader flags causing executable to use shared library from path(s) specified at compile time.

3.5.2 Version 4.1.0

- Arguments
Change in argument handling. Arguments to be passed to *makepkg* must now follow -. Arguments before the double dash are used by mkpkg itself. To keep backward compatibility the older *-mkp*- style arguments are honored, but the newer simpler ones are preferred. e.g. *-v*, *-verb* for verbose. Help available via *-h*.
- Config file now available.
Configs are looked for in */etc/mkpkg/config* then *~/.config/mkpkg/config*. It should be in TOML format. e.g. to change the default soname rebuild option:

```
soname_comp = "newer"
```

3.5.3 Version 4.0.0

- Soname drive rebuilds.
Adds support for detecting missing soname libraries, and triggering rebuild. If soname is found then no rebuild is done. Typically happens when older soname is deprecated.
- Adds new option *-mkp-refresh*.
Attempts to update saved metadata files. Faster, if imperfect, alternative to rebuild.